

Tarea 01: Código Ejecutado en Clase y Pruebas en Octave y Python

Maryom Alejandra Vicente González

Carné: 202202918

Escuela de Ingeniería Mecánica Eléctrica

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Resumen—En esta tarea se realizaron pruebas en GNU Octave y Python con una función seno, objetos numéricos, matrices, estructuras y operadores. Se compararon ambos lenguajes, observando resultados similares con algunas diferencias en su sintaxis.

```
plt.plot(x, y)
plt.show()
```

I. CÓDIGO

I-A. Repositorio

El código desarrollado durante la actividad se encuentra disponible en el siguiente repositorio:

<https://github.com/3050321780117-coder/Repopruebamaryom>

I-B. Función seno en Octave

El primer código ejecutado en clase consistió en la representación de una función seno desde 0 hasta 2π .

```
x = 0:0.01:2*pi;
y = sin(x);
plot(x, y)
```

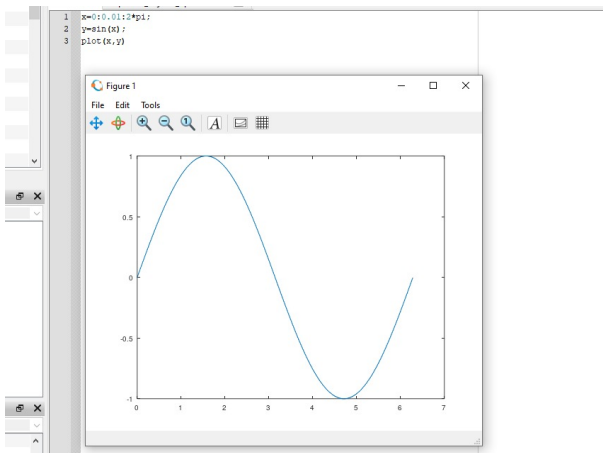


Figura 1. Código de la función seno ejecutado en Octave.

I-C. Función seno en Python

Se realizó el mismo procedimiento utilizando Python con las librerías NumPy y Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 2*np.pi, 0.01)
y = np.sin(x)
```

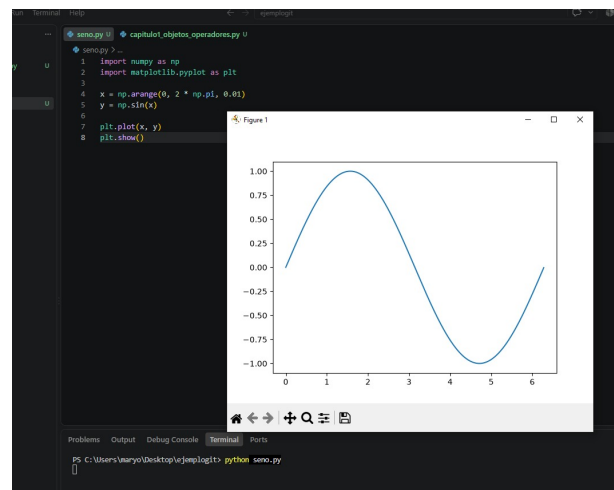


Figura 2. Código equivalente de la función seno en Python.

I-D. Objetos numéricos en Octave

Se probaron diferentes formas de representar valores numéricos.

```
375
3.75e2
3.75E2
0x17
```

I-E. Secuencias

En Octave se utilizó el operador dos puntos para generar diferentes secuencias numéricas.

```
1:10
1:1.6:10
```

I-F. Matrices

Se realizaron pruebas con matrices utilizando diferentes formas de declaración.

```
M = [1, 2, 3; 4, 5, 6; 7, 8, 9]
M = [1:4; 5:8]
```

I-G. Cadenas y estructuras

También se probaron cadenas de texto y estructuras en Octave.

```
'Cadena_de_string'
'A'

x = {}

x.secuencia = 1:5
x.matriz = [1,2;44,5]
x.string = 'Secuencia'

x.estructura = {}

x.estructura.numero = 0x177
x.estructura.letra = 'A'

x
```

I-H. Operadores aritméticos

Se utilizaron las operaciones básicas de suma, resta, multiplicación y división.

```
x = 2
y = 3

x + y
x - y
x * y
x / y

++x
--x
```

I-I. Operadores relacionales

Se probaron diferentes comparaciones entre dos valores.

```
x = 2
y = 3

x < y
x <= y
x == y
x > y
x >= y
x != y
```

I-J. Operadores lógicos

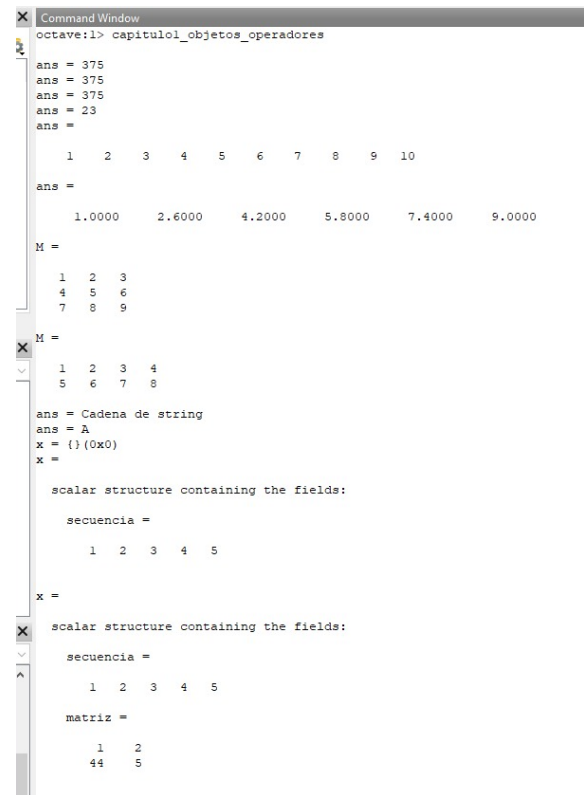
Finalmente se probaron operadores lógicos utilizando valores de uno y cero.

```
x = 1
y = 0

x & y
x | y
not (x)
```

I-K. Ejecución en Octave

Las siguientes capturas muestran la ejecución de los ejemplos realizados en GNU Octave.



```
octave:1> capitulo1_objetos_operadores

ans = 375
ans = 375
ans = 375
ans = 23
ans =

   1   2   3   4   5   6   7   8   9  10

ans =

   1.0000   2.6000   4.2000   5.8000   7.4000   9.0000

M =

   1   2   3
   4   5   6
   7   8   9

M =

   1   2   3   4
   5   6   7   8

ans = Cadena de string
ans = A
x = {} (0x0)
x =

scalar structure containing the fields:

  secuencia =

   1   2   3   4   5

x =

scalar structure containing the fields:

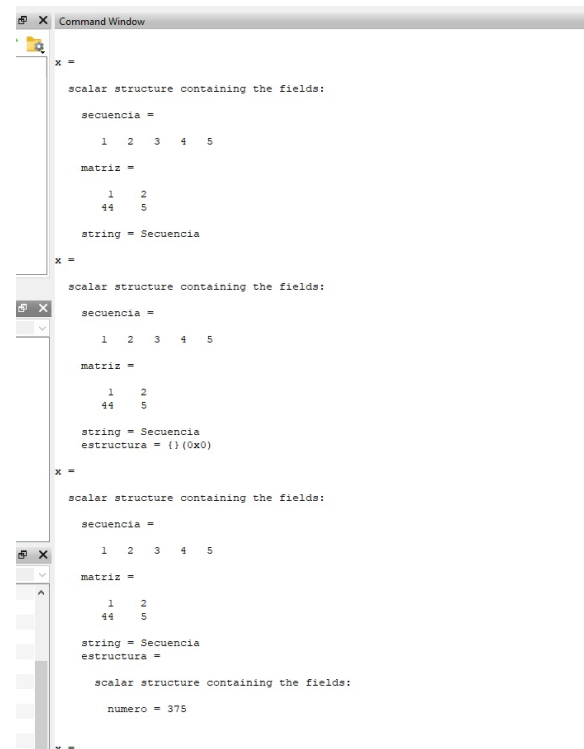
  secuencia =

   1   2   3   4   5

  matriz =

   1   2
  44   5
```

Figura 3. Pruebas realizadas en GNU Octave.



```
x =

scalar structure containing the fields:

  secuencia =

   1   2   3   4   5

  matriz =

   1   2
  44   5

  string = Secuencia

x =

scalar structure containing the fields:

  secuencia =

   1   2   3   4   5

  matriz =

   1   2
  44   5

  string = Secuencia
  estructura = {} (0x0)

x =

scalar structure containing the fields:

  secuencia =

   1   2   3   4   5

  matriz =

   1   2
  44   5

  string = Secuencia
  estructura =

scalar structure containing the fields:

  numero = 375

x =
```

Figura 4. Ejecución de objetos numéricos y secuencias.

```

x =
scalar structure containing the fields:

  secuencia =

    1 2 3 4 5

  matriz =

    1 2
   44 5

  string = Secuencia
  estructura =

  scalar structure containing the fields:

    numero = 375
    letra = A

x =
scalar structure containing the fields:

  secuencia =

    1 2 3 4 5

  matriz =

    1 2
   44 5

  string = Secuencia
  estructura =

  scalar structure containing the fields:

    numero = 375
    letra = A

x = 2
y = 3
ans = 5
ans = -1
ans = 6
ans = 0.6667
ans = 3
ans = 2
x = 2
y = 3
ans = 1

```

Figura 5. Ejecución de matrices y estructuras.

```

1 2
44 5

string = Secuencia
estructura =

scalar structure containing the fields:

  numero = 375
  letra = A

x = 2
y = 3
ans = 5
ans = -1
ans = 6
ans = 0.6667
ans = 3
ans = 2
x = 2
y = 3
ans = 1
ans = 1
ans = 0
ans = 0
ans = 0
ans = 1
x = 1
y = 0
ans = 0
ans = 1
ans = 0
octave:2> |

```

Figura 6. Ejecución de operadores en GNU Octave.

I-L. Equivalente en Python

Los ejemplos realizados en Octave fueron reproducidos en Python para comparar su funcionamiento y sintaxis.

```

seno.py U
capitulo1_objetos_operadores.py U
1 # =====
2 # CAPITULO 1 - OBJETOS NUMERICOS Y OPERADORES
3 # SU CODIGO EN PYTHON
4 # =====
5
6 import numpy as np
7
8
9 # =====
10 # OBJETOS NUMERICOS
11 # =====
12
13 print(375)
14 print(3.75e2)
15 print(3.75E2)
16 print(0x17)
17
18 # =====
19 # SECUENCIAS
20 # =====
21
22 print(list(range(1, 11)))
23
24 secuencia = np.arange(1, 10, 1.6)
25 print(secuencia)
26
27
28 # =====
29 # MATRICES
30 # =====
31
32
33 M = np.array([
34     [1, 2, 3],
35     [4, 5, 6],
36     [7, 8, 9]
37 ])
38
39 print(M)
40
41 M = np.array([
42     [1, 2, 3, 4],
43     [5, 6, 7, 8]
44 ])
45
46 print(M)
47

```

Figura 7. Código equivalente ejecutado en Python.

```

seno.py U  capitulo1_objetos_operadores.py U
capitulo1_objetos_operadores.py > ...
52
53 print("Cadena de string")
54 print("A")
55
56
57 # =====
58 # ESTRUCTURAS
59 # =====
60
61 x = {}
62
63 x["secuencia"] = [1, 2, 3, 4, 5]
64
65 x["matriz"] = [
66     [1, 2],
67     [44, 5]
68 ]
69
70 x["string"] = "Secuencia"
71
72 x["estructura"] = {}
73
74 x["estructura"]["numero"] = 0x177
75 x["estructura"]["letra"] = "A"
76
77 print(x)
78
79
80 # =====
81 # OPERADORES ARITMETICOS
82 # =====
83
84 x = 2
85 y = 3
86
87 print(x + y)
88 print(x - y)
89 print(x * y)
90 print(x / y)
91
92 x += 1
93 print(x)
94
95 x -= 1
96 print(x)
97
98
99 # =====

```

Figura 8. Pruebas de objetos numéricos y secuencias en Python.

```

96 print(x)
97
98
99 # =====
100 # OPERADORES RELACIONALES
101 # =====
102
103 x = 2
104 y = 3
105
106 print(x < y)
107 print(x <= y)
108 print(x == y)
109 print(x > y)
110 print(x >= y)
111 print(x != y)
112
113
114 # =====
115 # OPERADORES LOGICOS
116 # =====
117
118 x = True
119 y = False
120
121 print(x and y)
122 print(x or y)
123 print(not x)
124
125

```

Figura 9. Pruebas de matrices y estructuras en Python.

```

10 x -= 1
11 print(x)
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

Figura 10. Pruebas de operadores en Python.

II. RESULTADOS

Los códigos se ejecutaron correctamente en Octave y Python. En ambos casos se obtuvieron resultados similares en las operaciones, matrices, estructuras y en la gráfica de la función seno.

III. CONCLUSIONES

- Octave y Python permiten realizar operaciones y gráficas de forma parecida.
- Se practicó el uso de matrices, estructuras, secuencias y operadores en Octave.
- La principal diferencia observada entre ambos lenguajes fue la forma de escribir algunas instrucciones.